



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 4

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 25/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
3. Desarrollo	5
3.1. Ejercicio 1	5
3.2. Ejercicio 2	5
4. Resultados	10
4.1. Resultados del Ejercicio 1	10
4.2. Resultados del Ejercicio 2	11
5. Conclusiones	13

1. Introducción

1.1. Planteamiento del problema

Para esta practica contamos con dos ejercicios el primero nos solicitaba consultar con nuestro LLM y preguntarle ¿Qué es un destructor en java?, ¿Cómo funciona? y un ejemplo de uso para posteriormente comparar la respuesta de nuestro LLM entre todos los miembros del equipo. En el segundo ejercicio nos pide escribir una aplicación en java que determine e imprima el número de palabras duplicadas en una oración, pero considerando las siguientes condiciones: Considerar las mayúsculas y minúsculas como equivalentes. Ignorar los signos de puntuación y utilizando el método `AnalizadorPalabras`.

1.2. Motivación

Esta práctica busca que apliquemos los conceptos vistos en clase como el método constructor y la creación de objetos para dividir el problema no en tareas si no en modelos de objetos “físicos o simulados”.

1.3. Objetivos

- **General:** Aplicar los conceptos básicos de la programación orientada a objetos en un lenguaje de programación.
- **Específicos:**
 - Comprender el concepto de destructores en java mediante nuestro LLM.
 - Implementar algoritmos de manipulación y normalización de cadenas de texto (*strings*) que omitan signos de puntuación y diferencias de capitalización.
 - Encapsular la lógica de conteo de palabras duplicadas mediante el uso del método `AnalizadorPalabras` dentro de un modelo orientado a objetos.

2. Marco Teórico

1. Programación orientada a objetos: Se basa en la creación de objetos y la interacción con éstos, para el diseño y desarrollo de aplicaciones de computadora en la que los datos y los procedimientos para la manipulación de éstos están estrechamente ligados. [5]
2. Clase: Son plantillas para la creación de objetos. Define las variables y los métodos que son comunes para todos los objetos de un cierto tipo.[1]
3. Objeto: Representan cosas del mundo real, los elementos de un objeto se dividen en dos categorías principales atributos y métodos. [2]
4. Atributos: Los atributos, incluyen características sobre el objeto, por ejemplo, si consideramos un objeto Coche algunas de sus propiedades serán: el color, la marca, el modelo o el año de fabricación. [2]
5. Métodos: Los métodos definen las acciones que se pueden realizar con el objeto, por ejemplo, para el objeto Coche, los métodos podrían ser acelerar, frenar o girar. [2]
6. Instanciar: Una vez definida la clase se pueden crear objetos a partir de ésta, a este proceso se le conoce como crear instancias de una clase o instanciar una clase, cada uno de los objetos o instancias tiene su propia copia de las variables definidas en la clase de la cual son instanciados y comparten la misma implementación de los métodos. [4]
7. Sobrecarga de métodos (overload): La sobrecarga de métodos es la creación de varios métodos con el mismo nombre, pero con diferente lista de tipos de parámetros. Java utiliza el número y tipo de parámetros para seleccionar cuál definición de método ejecutar. [3]
8. Encapsulamiento: Sucede cuando un elemento del objeto (atributo o método) es envuelto en una capa protectora implica que los datos del objeto están protegidos o ocultos dentro de él , el programa no puede acceder de manera directa a los datos encapsulados de un objeto, esto se tiene que realizar con ayuda de los métodos del objeto. [6]
9. Constructor: Es un método que tiene el mismo nombre que la clase y cuyo propósito es inicializar los atributos de un nuevo objeto se ejecuta automáticamente cuando se crea un objeto o instancia de la clase. [4]
10. Mensajes: Los objetos entran en interacción enviándose mensajes entre ellos. Una vez que el objeto reciba un mensaje, actuará. La acción puede ser el envío de otros mensajes, el cambio de su estado, o la ejecución de toda otra tarea que el objeto deba realizar. Los objetos de un programa. interactúan y se comunican entre sí por medio de mensajes. [4]

3. Desarrollo

3.1. Ejercicio 1

El trabajo para este ejercicio se centró en colaborar con nuestro LLM preguntándole ¿Qué es un destructor en java?, ¿Cómo funciona? y un ejemplo de uso, preguntando esto nos respondió lo siguiente:

- En Java no hay destructores en el sentido tradicional, como sucede en lenguajes como C++. En vez de ello, Java emplea un sistema automático de gestión de memoria impulsado por el Recolector de Basura (Garbage Collector).
- Si en tu programa ya no hay ninguna referencia activa a un objeto (es decir, tu código ya no puede acceder a él), entonces el Recolector de Basura asume que es inútil y eventualmente cancela la asignación de memoria que ocupaba. No tienes que destruir el objeto manualmente ni controlas exactamente cuándo desaparecerá.
- Un ejemplo práctico es abrir y leer un archivo de tu disco duro: Tu programa abre un documento (como un Excel o un archivo de texto) para leer su contenido. Al hacerlo, el sistema operativo le pone un "candado" al archivo para evitar que otro programa lo corrompa o lo borre mientras tú lo estás leyendo. Si terminas de leer los datos y no "destruye" esa conexión, el archivo se queda bloqueado. Si el usuario intenta mover, borrar o editar ese documento desde otra aplicación, la computadora le mostrará el error: "El archivo está siendo usado por otro programa". Al usar el cierre automático de Java (try-with-resources), el lenguaje garantiza que en el milisegundo exacto en que terminas de leer el documento (o incluso si el programa falla a la mitad por un error), Java actúa como destructor y le avisa al sistema operativo: "Ya no necesito el archivo, quita el candado". Así, el archivo queda libre inmediatamente.

3.2. Ejercicio 2

Este ejercicio consta de escribir una aplicación en java que determine e imprima el número de palabras duplicadas en una oración pero cumpliendo las siguientes condiciones:

1. Considerar las mayúsculas y minúsculas como equivalentes.
2. Ignorar los signos de puntuación y utilizando el método `AnalizadorPalabras`.
 - Primero definimos la clase `AnalizadorPalabras`, esta clase encapsula la estructura de datos y los algoritmos necesarios para procesar las cadenas de texto.
 - `String oracion;` Es la variable que almacenará el texto ingresado por el usuario la declaramos como `private` aplicando el principio de encapsulamiento protegiendo el dato para que ninguna otra clase externa pueda modificarlo directamente por accidente.

- `HashMap<String, Integer> frecuencias;` Declara la estructura de datos que usamos para el conteo, los diamantes `< >` indican que las claves (las palabras) serán cadenas de texto (`String`) y los valores (las veces que se repiten) serán números enteros (`Integer`). Esta estructura es ideal para este problema porque nos permite asociar de forma eficiente una clave única (la palabra) con un valor numérico (su frecuencia de aparición).
- Creamos el constructor `AnalizadorPalabras(String oracion)`: Es el bloque de código especial que se ejecuta automáticamente al crear una instancia del objeto en la clase `Main`. Su función es inicializar el estado de la clase mediante dos acciones clave:
 1. Asigna la cadena recibida al atributo privado usando `this`, lo cual le indica al compilador que diferencie entre el parámetro que entra entre paréntesis y la variable interna protegida de la clase.
 2. Reserva el espacio en memoria dinámica para el mapa de dispersión `"this.frecuencias = new HashMap<>();"`. Este paso de inicialización es indispensable; de omitirse, la variable de la estructura valdría nulo y el programa colapsaría al intentar registrar la primera palabra.
- Para poder contar las palabras de manera precisa y cumpliendo la condición que nos solicitan (considerar las mayúsculas y minúsculas como equivalentes) creamos el método (`contarPalabras`) con el aplicamos una "limpieza" en la cadena de la siguiente manera:
- Utilizando `.toLowerCase()` estandarizamos todo el texto a minúsculas para que "Hola" y "hola" se cuenten como la misma palabra.
- Utilizamos `.replaceAll("[^a-záéíóúüñ0-9\\s]", "")` con una expresión regular (Regex) para eliminar todos los signos de puntuación y caracteres especiales del texto. Funciona reemplazando por una cadena vacía cualquier carácter que no coincida con los elementos permitidos definidos dentro de los corchetes.
 1. `^` (Negación): Al estar al principio de los corchetes, actúa como un operador lógico de negación. Indica que se debe buscar cualquier carácter que NO esté en la lista que sigue.
 2. `a-z`: Permite todas las letras minúsculas del alfabeto estándar. `áéíóúüñ`: Permite explícitamente los caracteres propios del idioma español, asegurando que las palabras con acentos, diéresis o la letra ñ no se rompan ni se eliminen.
 3. `0-9`: Permite cualquier dígito numérico.
 4. `\\s`: Es el símbolo en Regex para detectar espacios en blanco, tabulaciones o saltos de línea (la doble barra diagonal invertida se usa en Java para que el compilador lea correctamente el comando).
 5. Utilizamos `.trim()` para recortar los espacios en blanco que hayan quedado en los bordes de la oración (al principio y al final). Por ejemplo, si después de la limpieza tu texto quedó como `" hola mundo "`, el `.trim()` lo deja como `"hola mundo"`.

6. Y por último con `.split("\\s+")` corta la cadena de texto para dividirla y guardarla en un arreglo de palabras separadas. La cantidad de palabras es simplemente la longitud de este arreglo resultante.
- Para nuestro método (`obtenerNumeroPalabrasDuplicadas`) el funcionamiento es el siguiente:
 1. Primero, se vacía el mapa de frecuencias para evitar que se acumule basura o datos de operaciones previas.
 2. Se vuelve a normalizar el texto y, mediante un ciclo, se itera sobre cada palabra del arreglo.
 3. Lógica de inserción: Si la palabra ya existe en el mapa como clave, se recupera su valor actual y se le suma 1. Si es la primera vez que aparece, se inserta en el mapa inicializando su valor en 1.
 4. Al terminar, se realiza un segundo ciclo que evalúa únicamente los valores del mapa. Se utiliza un contador auxiliar que aumenta solo si encuentra un valor estrictamente mayor a 1, retornando así el total de palabras que se repiten.
 - Y por último para nuestro método (`mostrarDuplicadas`) el funcionamiento es el siguiente:
 1. Itera sobre las claves del mapa buscando aquellas cuyo valor de frecuencia sea mayor a 1 para imprimirlas.
 2. Se implementa una variable booleana (`hayPalabrasDuplicadas`) que inicia en falso, si el algoritmo encuentra al menos una coincidencia la bandera cambia a verdadero, si al terminar el ciclo la bandera sigue siendo falsa, el programa tiene la capacidad de informar al usuario que no hubo repeticiones, mejorando la experiencia de uso.
 - La Clase Principal (Main) actúa como el punto de entrada del programa y se encarga exclusivamente de la interacción con el usuario y la orquestación de los procesos.
 1. La lógica de captura de datos utiliza un ciclo que evalúa si la cadena ingresada está vacía (como presionar Enter sin escribir nada) y obliga a repetir la solicitud hasta obtener una entrada válida.
 2. Una vez validada la entrada, se instancia el objeto `AnalizadorPalabras`, a partir de ahí se manda a llamar a los métodos de conteo y visualización uno por uno, imprimiendo los resultados.
 3. Al finalizar las operaciones, se implementa el cierre del objeto `Scanner` para liberar el flujo de entrada de memoria.

FIGURA 1. CONCEPTUALIZACIÓN DE UN OBJETO 'ANALIZADORPALABRAS'

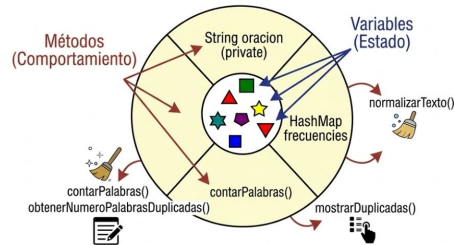


Figura 1. Conceptualización de un objeto 'AnalizadorPalabras' encapsulado.

Figura 1: Conceptualización.

FIGURA 2. MENSAJES ENTRE OBJETOS (ORQUESTACIÓN)

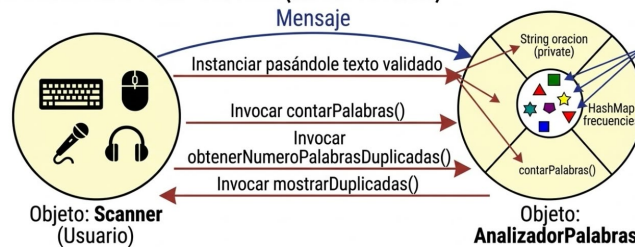


Figura 2. Mensajes entre objetos: 'Main' orquestando con 'Scanner' y 'AnalizadorPalabras'.

Figura 2: Mensajes.

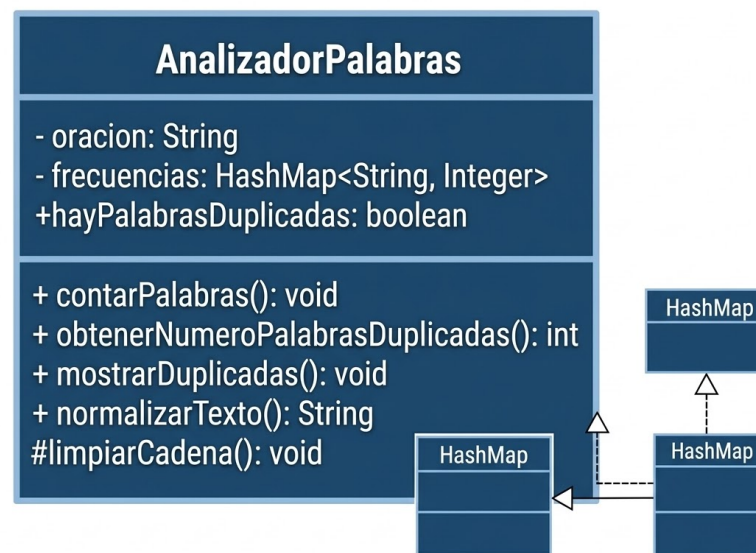


FIGURA 3. DIAGRAMA DE CLASE UML

Figura 3: Diagrama de clase.

Prueba 1: Como prueba, se ingresó el texto “EsTo Es UnA Prueba De UNA PruEBA” con repeticiones intencionales mezclando letras mayúsculas y minúsculas verificando

que se agruparan las palabras coincidentes (como "prueba" y "una") y que el mapa de frecuencias actualizara sus valores sin importar el formato original de escritura.

Prueba 2: Como prueba, se introdujo el texto "¡Ingeniería en computación, la mejor ingeniería!" con signos de admiración, comas y palabras acentuadas, comprobando que eliminaba los caracteres especiales de forma precisa.

Prueba 3: Se ingresó la oración " Prueba espacios prueba espacios " que contenía múltiples espacios en blanco consecutivos tanto al inicio, al final y entre palabras, demostrando que los métodos descartaban las separaciones extra, garantizando que el programa contabilizara únicamente las 4 palabras reales sin generar conteos vacíos.

Prueba 4: Se ingresó el texto "El código compila correctamente desde la terminal" el cual está completamente libre de repeticiones, verificando que la bandera booleana funcionaba adecuadamente.

Prueba 5: Como prueba se simuló un ingreso en blanco presionando la tecla Enter sin escribir ningún carácter , verificando que el ciclo do-while atrapara el error y mostrara la advertencia.

4. Resultados

4.1. Resultados del Ejercicio 1

Se muestra la respuesta obtenida al consultar al LLM sobre el funcionamiento del código.

En Java no existen los "destructores" tradicionales como en otros lenguajes (por ejemplo, C++). La gestión de la memoria y la eliminación de los objetos se realiza de forma 100% automática mediante un mecanismo interno llamado **Recolector de Basura (Garbage Collector)**.

- o **Funcionamiento del Recolector de Basura:** Mientras el programa se ejecuta, el Recolector de Basura rastrea qué objetos siguen en uso. Cuando un objeto ya no tiene ninguna variable o referencia apuntando hacia él (es decir, quedó "suelto" en la memoria), el sistema lo identifica como basura, lo destruye automáticamente y recupera ese espacio de memoria de forma silenciosa.
- o **El método `finalize()` (Obsoleto):** En versiones antiguas de Java, existía un método llamado `finalize()` que actuaba de forma similar a un destructor. Se mandaba llamar justo antes de que el objeto fuera eliminado. Sin embargo, como el Recolector de Basura trabaja a su propio ritmo, nunca se sabía exactamente en qué milisegundo se ejecutaría este método, volviéndolo ineficiente. A partir de Java 9, este método fue marcado como obsoleto (deprecated).
- o **La alternativa para liberar recursos:** Cuando un programador necesita cerrar o destruir algo de forma inmediata y obligatoria (como un archivo abierto o la conexión a una base de datos), Java implementa la interfaz `AutoCloseable` junto con la estructura `try-with-resources`. Esto asegura que los recursos se cierren y liberen en el instante exacto en que termina el bloque de código, simulando el control manual que ofrecería un destructor.

Figura 4: Respuesta del LLM.

¿Qué es un destructor en java?, ¿Cómo funciona? y un ejemplo de uso

En Java no existen los destructores en el sentido tradicional, como ocurre en lenguajes como C++. En su lugar, Java utiliza un sistema de gestión de memoria automático impulsado por el Recolector de Basura (Garbage Collector).

¿Cómo funciona la "destrucción" en Java?

Cuando un objeto ya no tiene ninguna referencia activa en tu programa (es decir, tu código ya no tiene forma de acceder a él), el Recolector de Basura asume que es inservible y, eventualmente, libera la memoria que ocupaba. Tú no tienes que destruirlo manualmente ni tienes control exacto sobre *cuándo* será eliminado.

El problema con los recursos externos:

El Recolector de Basura es excelente para la memoria RAM, pero es terrible para liberar recursos externos (archivos, conexiones a bases de datos, sockets de red). Como no sabes cuándo actuará, los recursos pueden quedarse abiertos indefinidamente.

(Nota histórica: Java solía tener un método llamado `finalize()` que intentaba actuar como un destructor, pero fue deprecado en Java 9. ⚠️ Se era impredecible, ralentizaba el programa y causaba bloqueos)

Figura 5: Respuesta del LLM.

4.2. Resultados del Ejercicio 2

Se muestra la ejecución del programa al ingresar el texto "EsTo Es UnA Prueba De UNA PruEBA" con repeticiones intencionales mezclando letras mayúsculas y minúsculas.

```
PS C:\Users\o-ale\OneDrive\Documentos\POO> java Ejercicio_2.Main
Ingresa una oración para analizar: EsTo Es UnA Prueba De UNA PruEBA

Resultados Retornados por el Analizador

Total de palabras: 7

Palabras duplicadas: 2
Palabras Duplicadas
una: 2 veces
prueba: 2 veces
PS C:\Users\o-ale\OneDrive\Documentos\POO>
```

Figura 6: Prueba con repeticiones intencionales mezclando letras mayúsculas y minúsculas.

Se muestra la ejecución del programa al ingresar el texto "¡Ingeniería en computación, la mejor ingeniería!" con signos de admiración, comas y palabras acentuadas.

```
PS> java Ejercicio_2.Main
Ingresa una oración para analizar: ¡Ingeniería en computación, la mejor ingeniería!

Resultados Retornados por el Analizador

Total de palabras: 6

Palabras duplicadas: 1
Palabras Duplicadas
ingeniera: 2 veces
PS C:\Users\o-ale\OneDrive\Documentos\POO>
```

Figura 7: Prueba con signos de admiración, comas y palabras acentuadas.

Se muestra la ejecución del programa al ingresar el texto "Prueba espacios prueba espacios" que contenía múltiples espacios en blanco consecutivos tanto al inicio, al final y entre palabras.

```
Ingresa una oración para analizar: Prueba espacios prueba espacios

Resultados Retornados por el Analizador

Total de palabras: 4

Palabras duplicadas: 2
Palabras Duplicadas
prueba: 2 veces
espacios: 2 veces
PS C:\Users\o-ale\OneDrive\Documentos\POO>
```

Figura 8: Prueba con múltiples espacios en blanco consecutivos tanto al inicio, al final y entre palabras.

Se muestra la ejecución del programa al ingresar el texto "El codigo compila correctamente desde la terminal" verificando el funcionamiento la bandera booleana.

```
Ingresa una oración para analizar: El codigo compila correctamente desde la terminal
Resultados Retornados por el Analizador
Total de palabras: 7
Palabras duplicadas: 0
Palabras Duplicadas
No hay palabras duplicadas
PS C:\Users\o-ale\OneDrive\Documentos\P00> |
```

Figura 9: Prueba sin repeticiones verificando el funcionamiento la bandera booleana.

Se muestra la ejecución del programa sin escribir ningún carácter

```
PS C:\Users\o-ale\OneDrive\Documentos\P00> java Ejercicio_2.Main
Ingresa una oración para analizar:
ERROR: no se ingresó nada, intenta de nuevo
Ingresa una oración para analizar: |
```

Figura 10: Prueba de error y advertencia.

5. Conclusiones

Durante el desarrollo de esta práctica, se logró aplicar con éxito los fundamentos de la Programación Orientada a Objetos (POO) en Java. Por un lado, la investigación del primer ejercicio permitió comprender la gestión de memoria y el ciclo de vida de los objetos en Java. La recolección de basura automatizada Garbage Collector simplifica el desarrollo al delegar la liberación de memoria al entorno de ejecución. Por otra parte el desarrollo de la aplicación de conteo en el segundo ejercicio el diseño de la clase **AnalizadorPalabras** permitió aislar la lógica de procesamiento y la estructura de datos (**HashMap**) de la interacción con el usuario gestionada en la clase **Main**. El uso de un **HashMap** demostró ser la estructura óptima para este problema, permitiendo asociar claves y actualizar sus frecuencias de repetición. La implementación combinada de expresiones regulares (**Regex**) y funciones de manipulación de cadenas (**toLowerCase**, **trim**, **split**) garantizó que el sistema interpretara la información correctamente. Las pruebas ejecutadas validó la robustez del código, demostrando que el programa es capaz de manejar espacios irregulares, omitir signos de puntuación, unificar mayúsculas y minúsculas, y prevenir excepciones en tiempo de ejecución al validar entradas vacías mediante el ciclo **do-while**.

Referencias

- [1] ¿Qué es una clase en Java? | Netec. (s. f.). Netec. <https://www.netec.com/que-es-una-clase-en-java>
- [2] Sandoval, E. (2023, 21 septiembre). ¿Qué es un objeto en programación? Ebac. <https://ebac.mx/blog/objeto-en-programacion>
- [3] Sobrecarga de metodos y de constructores. (s.f.). http://profesores.fi-b.unam.mx/carlos/java/java_basico4_6.html
- [4] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.
- [5] Introducción a la programación con Java (1.a ed.). (2009). Mc Graw Hill.
- [6] Olguín, J. A. S. G. R. A. S. (s. f.). Abstracción y encapsulamiento. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html